

Módulo: Programación Multimedia y dispositivos móviles. Unidad de Trabajo N°1: Introducción al desarrollo de aplicaciones móviles.

Actividad de Clase 8: RecyclerView.

1. Objetivo general.

Conocer, manejar y generar aplicaciones que contengan RecyclerView.

2. Metodología.

Cada alumno realizará la app.

3. Descripción.

Introducción: ¿Qué son los RecyclerView?

RecyclerView, es un control que nos ayudará a crear controles de tipo lista y tabla. Este control llegó con Android 5.0 como una mejor alternativa a los controles ListView (lista) y GridView (tabla) de versiones anteriores (ya no se han utilizan), aportando en la mayoría de ocasiones flexibilidad de sobra para suplir la funcionalidad de ambos, e ir incluso más allá.

RecyclerView nos va a permitir mostrar en pantalla colecciones de datos. Pero lo va a hacer de una forma algo distinta a la que utilizamos con los controles anteriores. Y es que RecyclerView no va a hacer «casi nada» por sí mismo, sino que se va a sustentar sobre otros componentes complementarios para determinar cómo acceder a los datos y cómo mostrarlos. Los más importantes serán

- **RecyclerView.Adapter**
- **RecyclerView.ViewHolder**
- **LayoutManager**
- **ItemDecoration**
- **ItemAnimator**

Como en el caso del **Spinner**, un **RecyclerView** se apoya también en un adaptador para trabajar con nuestros datos, en este caso un adaptador que herede de la clase **RecyclerView.Adapter**. La peculiaridad en esta ocasión es que este tipo de adaptador utilizará internamente el patrón **view holder** que dotará de una mayor eficiencia al control, y de ahí la necesidad del segundo componente de la lista anterior, **RecyclerView.ViewHolder**.

En resumen, un view holder se encargará de de contener y gestionar las vistas o controles asociados a cada elemento individual de la lista. El control **RecyclerView** se encargará de crear tantos view holder como sea necesario para mostrar los elementos de la lista que se ven en pantalla y los gestionará eficientemente de forma que no tenga que crear nuevos objetos para mostrar más elementos de la lista al hacer scroll, sino

que tratará de «reciclar» aquellos que ya no sirven por estar asociados a otros elementos de la lista que ya han salido de la pantalla.

Una vista de tipo **RecyclerView** no es como los antiguos **ListView** y **GridView**. Por el contrario no determina por sí sola la forma en que se van a mostrar en pantalla los elementos de nuestra colección, sino que va a delegar esa tarea a otro componente llamado **LayoutManager**, que también tendremos que crear y asociar al **RecyclerView** para su correcto funcionamiento. Por suerte, el SDK incorpora de serie algunos **LayoutManager** para las representaciones más habituales de los datos: lista vertical u horizontal (**LinearLayoutManager**), tabla tradicional (**GridLayoutManager**) y tabla apilada o de celdas no alineadas (**StaggeredGridLayoutManager**). Por tanto, siempre que optemos por alguna de estas distribuciones de elementos no tendremos que crear nuestro propio **LayoutManager** personalizado, aunque por supuesto nada nos impide hacerlo, y ahí uno de los puntos fuertes de este componente: su flexibilidad.

No siempre será obligatorio implementar todos estos componentes para hacer uso de un **RecyclerView**. Lo más habitual será implementar el **Adapter** y el **ViewHolder**, utilizar alguno de los **LayoutManager** predefinidos, y sólo en caso de necesidad crear los **ItemDecoration** e **ItemAnimator** necesarios para dar un toque de personalización especial a nuestra aplicación.

Creación de un aplicación que use RecyclerView.

Realiza lo siguiente:

1. Para ver el uso del control **RecyclerView**, vamos a crear una aplicación de ejemplo que mostrará una lista de elementos que estará formada por dos líneas de texto.
2. Crea un nuevo Proyecto.
3. Tras esto ya podremos añadir un nuevo **RecyclerView** al layout de nuestra actividad principal. En la siguiente imagen puedes ver un ejemplo, realizado con Linear Layout.

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    tools:context=".MainActivity">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="RecyclerView" />

    <androidx.recyclerview.widget.RecyclerView
        android:id="@+id/recview"
        android:layout_width="match_parent"
        android:layout_height="wrap_content" />

</LinearLayout>
```

4. En el siguiente paso, crearemos una clase que encapsulará los dos datos que compondrán cada elemento a mostrar en la tabla. La llamaremos Equipos. Para hacerlo vamos a utilizar una data class de Kotlin.

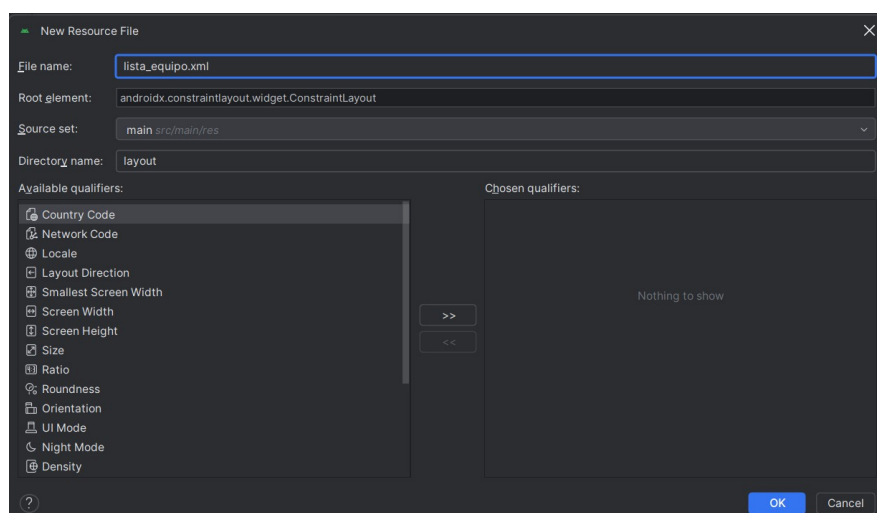
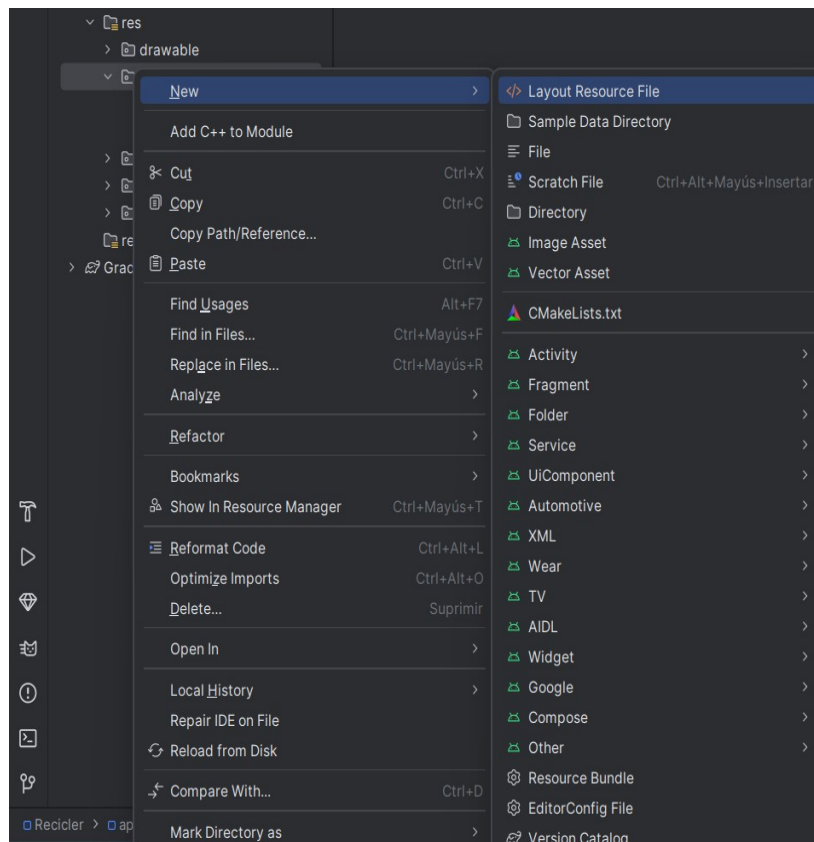
¿Qué es una Data Class en Kotlin?

5. Es una clase que contiene solamente atributos que quedemos guardar, por ejemplo datos de un Equipo. Con esto conseguimos por ejemplo no tener varias variables «Equipo», «njugadores» para almacenar todos los Equipos con los que vamos a trabajar.
6. Para ello creamos una nueva clase en **Kotlin** a la que llamaremos Equipos. En la siguiente imagen puedes ver un ejemplo:

```
data class Equipos(val nombre: String, val njugadores: Int)
```

7. En cada elemento de la lista queremos mostrar ambos datos, por lo que el siguiente paso será crear un layout XML con la estructura visual que deseemos. En este caso voy a mostrarlos en dos etiquetas de texto (TextView), la primera de ellas en **negrita** y con un tamaño de letra un poco mayor. Llamaremos a este layout «**lista_equipo.xml**» y lo colocaremos en la carpeta /app/res/layout:
8. Para ello sobre la carpeta layout dentro de res le das a botón derecho y en New

9. Layout Resource File



10. Ahora creamos el layout que será un LinearLayout de orientación vertical con dos TextView. A continuación puedes observar un ejemplo:

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
3     android:layout_width="match_parent"
4     android:layout_height="wrap_content"
5     android:orientation="vertical">
6
7     <TextView
8         android:id="@+id/tvNombre"
9         android:layout_width="match_parent"
10        android:layout_height="match_parent"
11        android:textSize="30sp"
12        android:textStyle="bold" />
13
14    <TextView
15        android:id="@+id/tvJugadores"
16        android:layout_width="match_parent"
17        android:layout_height="match_parent"
18        android:textSize="20sp"
19        android:textStyle="italic" />
20 </LinearLayout>
```

11. Ahora vamos escribir nuestro adaptador. Dicho adaptador debe heredar (extender) de la clase **RecyclerView.Adapter**, donde tendremos que sobrescribir (override) principalmente tres métodos:

12. **onCreateViewHolder()**. Encargado de crear los nuevos objetos ViewHolder necesarios para los elementos de la colección.

13. **onBindViewHolder()**. Encargado de actualizar los datos de un ViewHolder ya existente.

14. **onItemCount()**. Indica el número de elementos de la colección de datos.

15. Para nuestra aplicación de ejemplo utilizaré como fuente de datos un simple array de objetos de tipo Equipos.

16.

17. La clase **RecyclerView.Adapter** nos obligará de cierta forma a hacer uso del patrón view holder. Por tanto, para poder seguir con la implementación del adaptador lo primero que definiremos será el **ViewHolder** necesario para nuestro caso de ejemplo. Lo definiremos como clase interna a nuestro adaptador, extendiendo de la clase **RecyclerView.ViewHolder**, tan sólo tendremos que incluir como atributos las referencias a los controles del layout de un elemento de la lista (en nuestro caso los dos TextView) e inicializarlas en el constructor utilizando como siempre el método **findViewById<TextView>()** sobre la vista recibida como parámetro. Por comodidad añadiremos también un método auxiliar, que llamaremos **bindEquipo()**, que se encargue de asignar los contenidos de los dos cuadros de texto a partir de un objeto cuando nos haga falta. A continuación puedes observar un ejemplo de una implementación de un **ViewHolder** en este caso se llama **EquiposViewHolder**

```
class AdaptadorEquipos(private val datos: Array<Equipos>): RecyclerView.Adapter<AdaptadorEquipos.EquiposViewHolder>() {  
  
    class EquiposViewHolder(val item: View): RecyclerView.ViewHolder(item) {  
        val tvNombre = item.findViewById<TextView>(R.id.tvNombre)  
        val tvJugadores = item.findViewById<TextView>(R.id.tvJugadores)  
  
        fun bindEquipo(equipo: Equipos) {  
            tvNombre.text = equipo.nombre  
            tvJugadores.text = equipo.njugadores.toString()  
        }  
    }  
  
}
```

18. Una vez finalizado nuestro **ViewHolder** podemos ya seguir con la implementación del adaptador sobrescribiendo los métodos indicados. En el método **onCreateViewHolder()** nos limitaremos a inflar (construir) una vista a partir del layout correspondiente a los elementos de la lista (**lista_equipo**), y crear y devolver un nuevo ViewHolder llamando al constructor de nuestra clase EquiposViewHolder pasándole dicha vista como parámetro.

19. Los dos métodos restantes son aún más sencillos. En **onBindViewHolder()** tan sólo tendremos que recuperar el objeto Equipos correspondiente a la posición recibida como parámetro y asignar sus datos sobre el ViewHolder también recibido como parámetro. Por su parte, **getItemCount()** tan sólo devolverá el tamaño del array de datos. En la siguiente imagen puedes ver los tres métodos y luego puedes ver una imagen con la clase AdaptadorEquipos completa.

```
override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): EquiposViewHolder {  
    val item = LayoutInflater.from(parent.context).inflate(R.layout.lista_equipo, parent, attachToRoot: false) as LinearLayout  
    return EquiposViewHolder(item)  
}  
  
override fun onBindViewHolder(holder: EquiposViewHolder, position: Int) {  
    val equipo = datos[position]  
    holder.bindEquipo(equipo)  
}  
  
override fun getItemCount(): Int = datos.size
```

```
class AdaptadorEquipos(private val datos: Array<Equipos>): RecyclerView.Adapter<AdaptadorEquipos.EquiposViewHolder>() {

    class EquiposViewHolder(val item: View): RecyclerView.ViewHolder(item) {
        val tvNombre = item.findViewById<TextView>(R.id.tvNombre)
        val tvJugadores = item.findViewById<TextView>(R.id.tvJugadores)

        fun bindEquipo(equipo: Equipos) {
            tvNombre.text = equipo.nombre
            tvJugadores.text = equipo.njugadores.toString()
        }
    }

    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): EquiposViewHolder {
        val item = LayoutInflater.from(parent.context).inflate(R.layout.lista_equipo, parent, attachToRoot: false) as LinearLayout
        return EquiposViewHolder(item)
    }

    override fun onBindViewHolder(holder: EquiposViewHolder, position: Int) {
        val equipo = datos[position]
        holder.bindEquipo(equipo)
    }

    override fun getItemCount(): Int = datos.size
}
```

20. Ya tendríamos terminado nuestro adaptador. Ahora debemos asignarlo al **RecyclerView** en la actividad principal. Para ello creamos nuestro adaptador personalizado **AdaptadorEquipos** pasándole como parámetro la lista de datos y asignarlo al control RecyclerView utilizando su propiedad **adapter**. Antes de asociar el adaptador al RecyclerView, le hemos creado y asignado un **LayoutManager** para determinar la forma en la que se distribuirán los datos en pantalla. Si lo que queremos es mostrar los datos en forma de lista o tabla (al estilo de los antiguos **ListView** o **GridView**) no tendremos que implementar nuestro propio **LayoutManager**, ya que el SDK proporciona varias clases predefinidas para los tipos más habituales. Nosotros queremos mostrar los datos en forma de lista con desplazamiento vertical. Para ello tenemos disponible la clase **LinearLayoutManager**, por lo que tan sólo tendremos que instanciar un objeto de dicha clase indicando en el constructor la orientación del desplazamiento (**LinearLayoutManager.VERTICAL** o **LinearLayoutManager.HORIZONTAL**) y lo asignamos al RecyclerView mediante su propiedad **layoutManager**. A continuación puedes observar un ejemplo del código añadido en **MainActivity.kt** para trabajar con RecyclerView.

```
// Accedemos al RecyclerView
val reci = findViewById<RecyclerView>(R.id.recview)

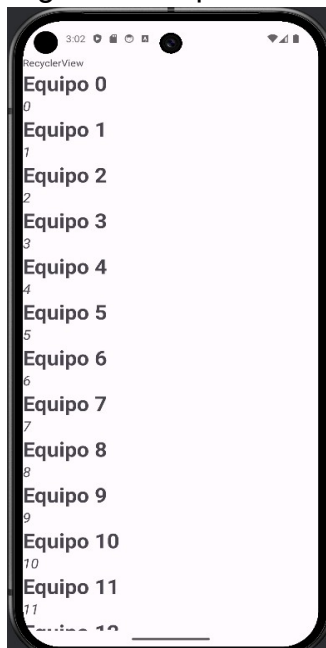
// Creamos el array con 50 utilizando un función Lambda
val datos = Array( size: 50) { i -> Equipos( nombre: "Equipo $i", i)}

// Creamos el adaptador
val adaptador = AdaptadorEquipos(datos)

// Especificamos el LayoutManager que vamos a utilizar, en este caso LinearLayout vertical
reci.layoutManager = LinearLayoutManager( context: this, LinearLayoutManager.VERTICAL, reverseLayout: false)

// Asociamos al RecyclerView el adaptador
reci.adapter = adaptador
```

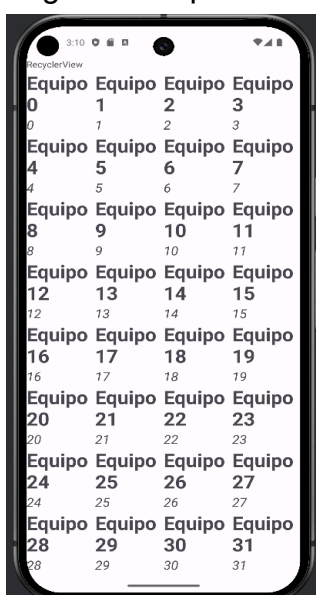
21. Ahora vamos a ver una imagen de la aplicación en funcionamiento.



22. Si quisiéramos mostrar los datos de forma tabular tan sólo tendríamos que cambiar la asignación del `LayoutManager` anterior y utilizar un `GridLayoutManager`, al que pasaremos como parámetro el número de columnas a mostrar. Modificaríamos `recView.layoutManager` como aparece en la siguiente imagen:

```
reci.layoutManager = GridLayoutManager(context: this, spanCount: 4)
```

23. La aplicación quedaría con el siguiente aspecto similar al siguiente:



Responder a eventos.

24. Nos falta como responder a los eventos que se produzcan sobre el **RecyclerView**, como opción más habitual el evento click sobre un elemento de la lista. la clase RecyclerView no incluye un evento **onItemClick** como ocurre en el caso de ListView. RecyclerView delega también esta tarea a otro componente, en este caso a la propia vista que conforma cada elemento de la colección.
25. La vista a la que tendremos que asociar el código a ejecutar como respuesta al evento click. Esto podemos hacerlo de diferentes formas, voy a mostrar una de ellas. En nuestro caso aprovecharemos el método **onBindViewHolder()** para asignar a su vista asociada el evento **onClick** (marcado en rojo en la imagen). Para poder asignar el evento desde fuera del adaptador, incluiremos la función a llamar como nuevo atributo del adaptador (marcado en rojo en la imagen). Adicionalmente, por comodidad, pasaremos como parámetro de esta función el dato (en nuestro ejemplo el objeto Equipos) contenido en ese elemento. Vamos a ver en la siguiente imagen el código primero veremos los cambios en el adaptador:

```
class AdaptadorEquipos(private val datos: Array<Equipos>, private val clickListener: (Equipos) -> Unit) : RecyclerView.Adapter<AdaptadorEquipos.EquiposViewHolder>() {

    class EquiposViewHolder(val item: View): RecyclerView.ViewHolder(item) {
        val tvNombre = item.findViewById<TextView>(R.id.tvNombre)
        val tvJugadores = item.findViewById<TextView>(R.id.tvJugadores)

        fun bindEquipo(equipo: Equipos) {
            tvNombre.text = equipo.nombre
            tvJugadores.text = equipo.jugadores.toString()
        }
    }

    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): EquiposViewHolder {
        val item = LayoutInflater.from(parent.context).inflate(R.layout.lista_equipo, parent, attachToRoot = false) as LinearLayout
        return EquiposViewHolder(item)
    }

    override fun onBindViewHolder(holder: EquiposViewHolder, position: Int) {
        val equipo = datos[position]
        holder.bindEquipo(equipo)

        holder.item.setOnClickListener { clickListener(equipo) }
    }

    override fun getItemCount(): Int = datos.size
}
```

26. Y ahora en la siguiente imagen puedes ver la clase MainActivity.kt donde cada vez que se pulse mostramos un mensaje en el Toast

```
// Creamos el adaptador y asociamos la función para que cada vez que se haga click aparezca un mensaje en el Toast
val adaptador = AdaptadorEquipos(datos) {
    Toast.makeText(context, "Has seleccionado -> ${it.nombre}", Toast.LENGTH_LONG).show()
}
```

27. A continuación vemos el funcionamiento de la aplicación.



28. En este caso al pulsar sobre un elemento de la lista tan sólo aparecerá un mensaje en el Toast

ItemDecoration.

29. Los **ItemDecoration** nos servirán para personalizar el aspecto de un **RecyclerView** más allá de la distribución de los elementos en pantalla. El ejemplo típico de esto son los separadores o divisores de una lista.

30. Crear un **ItemDecoration** personalizado no es una tarea demasiado fácil, o al menos no inmediata, por lo que por ahora no lo vamos a realizar. Pero para el caso de los separadores de lista Android ya nos proporciona de serie la clase **DividerItemDecoration**. Para utilizar este componente deberemos simplemente crear el objeto y asociarlo a nuestro **RecyclerView** mediante `addItemDecoration()` ahora podemos observar un ejemplo de código en nuestra actividad principal que estamos utilizando como ejemplo:

```
// Creamos el ItemDecoration en ese caso Vertical
reci.addItemDecoration(DividerItemDecoration(context, this, DividerItemDecoration.VERTICAL))
```

31. Ahora vemos en la siguiente imagen el aspecto del programa:

